Relatório de auditoria técnica: K.E.R.N.E.L 0.5.0

Data: 14 de outubro de 2026
Versão auditada: 0.5.0, commit `3548c0b` (ramo `main`, sem alterações pendentes no momento da auditoria)
Ambiente: Windows 10, i5-2400, 12 GB de RAM, RX 570 4 GB, Python 3.12.10
Auditor: Claude (Claude Code), a pedido de Henrique Lima

Como a auditoria foi feita

Nenhum arquivo de código foi alterado. O trabalho foi só de leitura e de execuções que não mudam nada:

- leitura do código-fonte e buscas por todos os pontos que executam programas, gravam arquivos, abrem conexões de rede ou desserializam dados;
- leitura dos bancos de dados em modo somente leitura (`mode=ro`), do registro (`dados/registro/*.jsonl`) e do log do serviço;
- consulta, sem alterar nada, de quem escuta na porta 8765, do perfil da rede e das regras de firewall;
- execução da suíte de testes inteira (os testes trabalham em pastas temporárias) e da avaliação do cérebro (`python -m kernel cerebro avaliar`, que só lê).

O único arquivo criado foi este relatório (e o PDF dele), na pasta `docs/`.

Resumo

O K.E.R.N.E.L é um assistente local que funciona e está bem coberto por testes: **332 testes, 0 falhas, 1 pulado**. A arquitetura é a do plano: um núcleo com laço de autonomia que consulta órgãos trocáveis, e toda ação passa pela segurança.

- **Não consegue mudar o próprio código.** Nenhum caminho do serviço grava na pasta `kernel/`. Ele muda a própria memória (sempre) e os próprios pesos (só com o treino que você aprova).
- **Sozinho, só lê.** Não escreve, não move e não apaga arquivo seu. Toda ação no PC (abrir programa, abrir página, baixar a Wikipédia, treinar) pede confirmação.
- **O freio funciona e é testado:** "para", o botão de pausa, o desligar e o modo seguro.
- **O cérebro não passou do portão da fase 1.** Escreve português com gramática, mas sem sentido: perda de validação 3,26, perplexidade 26. Leu cerca de 22% dos dados uma vez.

Os achados mais importantes:

#	Severidade	Achado
A1	Média	O selo de integridade cobre 4 arquivos (`kernel/seguranca/`). Código que também aplica regras (a autenticação da API, o freio "para", as ferramentas, o navegador, o explorador) fica fora dele.
A2	Média	Com `[celular] ligado = true`, o serviço escuta em `0.0.0.0:8765` por HTTP, sem criptografia. Hoje o firewall provavelmente barra a entrada (rede "Pública", sem regra para o Python).
A3	Média	Uma sessão de treino de 6 horas, aprovada em 13/10 às 23:32, terminou depois de 12 minutos, sem erro e sem motivo registrado. A saída do processo de treino não é gravada em lugar nenhum.
A4	Baixa a média	Privacidade: com `assuntos_dos_arquivos = true` (sua configuração), palavras tiradas dos seus documentos saem do PC como termos de busca. A memória guarda trechos dos seus documentos sem criptografia.
A5	Baixa	O treino carrega o checkpoint com `torch.load(weights_only=False)`, que executaria código embutido num checkpoint adulterado.
A6	Baixa	`pypdf` e `soundfile` são usados, mas não estão em nenhum `requirements*.txt`.

#	Severidade	Achado
A7	Baixa	Não há cópia de segurança automática dos bancos, nem rotação do registro e do log.
A8	Baixa	Falhas de entendimento vistas no uso real: "Treine por 6 horas" (sem a palavra "cérebro"), "como foi seu último treino?" e "O que é o Brasil?" (respondeu sobre um álbum).

A seção 9 traz um plano em degraus para o autoaperfeiçoamento controlado. A regra central: o K.E.R.N.E.L pode **propor e avaliar** melhorias, mas nada chega ao uso sem passar por provas automáticas e pela sua aprovação. O núcleo de segurança fica fora do alcance de qualquer mudança automática.

---

1. Arquitetura real

1.1 Processos e linhas de execução

```
pythonw -m kernel iniciar  (um processo)
├── servidor HTTP (uvicorn + FastAPI)  ← janela de chat e celular
│     porta 8765; 127.0.0.1, ou 0.0.0.0 se [celular] ligado = true
├── laço principal (asyncio)
│     recebe as suas mensagens: reflexos → diálogo → classificador → órgãos
├── laço de autonomia (uma thread, uma "volta" a cada 5 s)
│     perceber → lembrar → avaliar (impulsos) → decidir → agir → refletir
│     ações possíveis sozinho: explorar pastas, pesquisar na internet,
│     arrumar a memória, contar novidades, perguntar, escrever o diário
├── ouvidos (thread de microfone; hoje desligados: não há modelo treinado)
└── estudo (thread) → subprocesso "python -m kernel cerebro treinar"
                      (o PyTorch roda fora do serviço)
```

O serviço não carrega o PyTorch. O cérebro é usado no dia a dia por uma cópia da rede escrita em NumPy (`cerebro/inferencia.py`).

1.2 Pacotes

Pacote	Linhas	O que faz
`nucleo/`	1.859	Núcleo (`servico.py`), reflexos, regras de diálogo, modelo de si, trabalhos longos (estudo e treino)
`voz/`	1.897	Pronúncia, alinhador forçado, marcas de tom, PSOLA, banco de difones, sintetizador de formantes
`cerebro/`	1.446	micrograd, bigramas, BPE, GPT, AdamW, treino, inferência em NumPy, dados da Wikipédia
`internet/`	1.041	Navegador próprio (só leitura), HTML para texto, fontes (Wikipédia, Wikcionário, buscador), pesquisador
`exploracao/`	736	Mapa de pastas e arquivos, leitores (texto, Office, PDF), explorador, descrição da máquina
`ouvidos/`	525	Áudio, MFCC, rede da palavra de ativação, gravação e escuta
`seguranca/`	436	Níveis e pedidos de confirmação, regras de exploração e navegação, pareamento do celular, selo
`comandos/`	424	Classificador de intenções (regressão softmax), extração de argumentos, calculadora
`maos/`	327	Ferramentas: abrir programa, abrir página, listar pasta, anotar lembrete
`memoria/`	326	SQLite com índice invertido e BM25 escrito à mão

Pacote	Linhas	O que faz
`conhecimento/`	253	Enciclopédia local (Wikipédia) e resumo por extração
`interface/`	247	API (FastAPI) com o porteiro de autenticação
`mente/`	222	Impulsos e diário
raiz (`config`, `registro`, `__main__`...)	881	Configuração, registro de eventos, linha de comando
Total	**10.620**	mais 1.057 linhas de HTML, CSS e JavaScript da interface

Os testes têm 3.599 linhas em 20 arquivos.

1.3 Bibliotecas

Quase tudo o que é "inteligência" foi escrito no projeto: BM25, classificador, tokenizador BPE, GPT, AdamW, micrograd, MFCC, alinhador, PSOLA, leitor de HTML, resumo. As bibliotecas de fora fazem contas, rede e formatos de arquivo:

Biblioteca	Versão	Onde	Para quê
numpy	2.5.4	14 arquivos	Álgebra: classificador, inferência do cérebro, voz, ouvidos
torch	2.4.1	`cerebro/` (5 arquivos)	Derivadas automáticas e GPU, só no treino
torch-directml	0.2.5	`cerebro/dispositivo.py`	Treino na RX 570
tensorboard	2.21.0	`cerebro/treinar.py` (opcional)	Gráficos de treino
fastapi / starlette / pydantic	0.143 / 1.7 / 2.14	`interface/api.py`	API HTTP e validação das mensagens
uvicorn	0.54.0	`__main__.py`	Servidor HTTP
pypdf	6.20.1	`exploracao/leitores.py`	Texto de PDFs
soundfile	0.14.0	`voz/corpus.py`	Ler os MP3 do corpus de voz
sounddevice	0.5.6	`ouvidos/`	Microfone e alto-falante
pytest	9.1.1	testes	Testes

O banco de dados é o SQLite da biblioteca padrão do Python.

Achado A6: `pypdf` e `soundfile` não estão em nenhum `requirements*.txt`. Numa instalação nova pelo `instalar.ps1`, os PDFs entrariam no mapa só pelo nome, e montar o banco de voz falharia.

1.4 Dados em disco (`dados/`, fora do Git)

Arquivo	Tamanho	Conteúdo
`cerebro/`	6,5 GB	Corpus da Wikipédia (2,4 GB), tokens de treino (771 milhões), checkpoint, pesos
`conhecimento.db`	2,1 GB	Enciclopédia local: 647.741 artigos
`voz/`	233 MB	Corpus e banco de difones
`copia-antes-da-v0.5/`	72 MB	Cópia manual dos bancos, feita antes da faxina da v0.5
`kernel.db`	54 MB	Memória: 1.450 trechos de pesquisa, 173 de leitura, 15 conversas, 5 resumos, 4 diários, 2 fatos

Arquivo	Tamanho	Conteúdo
`mapa.db`	32 MB	Mapa: 127 pastas, 6.612 arquivos (24 lidos)
`registro/`	5,1 MB	Registro de eventos: 22.833 linhas em 3 dias
`servico.log`	2,2 MB	Saída do serviço (tracebacks)
`internet.db`	132 KB	Fila de pesquisa: 66 assuntos estudados, 374 na fila, 164 páginas lidas

2. Estado do treinamento do cérebro

2.1 O modelo

GPT com 6 camadas, 6 cabeças de atenção, dimensão 192, contexto de 256 pedaços de palavra e vocabulário de 4.096 (BPE treinado no projeto): **3,5 milhões de parâmetros** (configuração "mini").

2.2 O que ele já treinou

- **Passos:** 20.573, em 6,9 horas de GPU, a cerca de 6.890 tokens por segundo. Cada passo vê 8.192 tokens (lote 16, acúmulo 2, contexto 256).
- **Tokens lidos:** cerca de 170 milhões, de 771 milhões disponíveis. É **22% de uma passada** pelos dados.

A curva de validação mostra a queda rápida no começo e o platô depois:

Passo	Perda de treino	Perda de validação	Taxa de aprendizado
500	5,13	5,10	6,0e-4
2.500	3,99	4,05	5,8e-4
5.000	3,57	3,66	5,3e-4
10.000	3,23	3,41	3,3e-4
15.000	3,10	3,19	1,4e-4
20.000	3,15	**3,16** (melhor)	6,0e-5
20.573	3,22	3,30	3,0e-4

Os dois últimos pontos são a sessão de 13/10, que recomeçou com a taxa mais alta. Uma subida passageira da perda é o esperado nesse caso. O serviço continua usando os pesos do passo 20.000: só um resultado melhor substitui o `modelo.npz`.

2.3 Medida feita nesta auditoria

Em 10.240 tokens de texto que ele nunca viu (40 janelas de `val.bin`), a **perda foi 3,259 e a perplexidade, 26,0**. A cada pedaço de palavra, ele hesita entre umas 26 opções; um chute às cegas hesitaria entre 4.096.

Três amostras:

> *O Brasil é uma instituição de fundação fundada em 1994, que foi criada a instituição, criadora e produtor executivo...*
>
> *A inteligência artificial é a única classe, com o tempo de auge a ter uma taxa em que a máquina se converte para um único método...*
>
> *Em 1500, o MAR foi um dos primeiros fundos de mestre de Bolonha (1915)...*

Conclusão: a gramática e o estilo da Wikipédia foram aprendidos; o sentido, não. O portão da fase 1 ("gera frases coerentes em português") não foi atingido. Para esse tamanho, isso é o esperado. Mais sessões devem

baixar a perda um pouco (algo como 3,0 a 3,1), mas não resolvem a falta de sentido; o salto é o cérebro da fase 4.

2.4 Como o serviço usa o cérebro

A inferência é em NumPy, com memória de chaves e valores, e gera de 100 a 200 tokens por segundo. A amostragem usa temperatura, `top_k`, `top_p`, penalidade de repetição e bloqueio de trincas repetidas.

Na conversa normal o cérebro fica de fora (`[cerebro] na_conversa = false`). Ele aparece com "completa: ..." e na resposta a "como está o seu cérebro?". Ele **não chama ferramentas e não gera código**: o texto dele só é mostrado.

2.5 Anomalia no treino (achado A3)

Em 13/10, às 23:32, você aprovou "Treinar meu cérebro por 6 horas". O processo terminou às 23:44, no passo 20.573, com saída normal ("Treino encerrado..."), sem erro e sem mensagem sua pedindo para parar.

O laço de treino só termina sozinho por quatro motivos: o tempo acabou, a meta de passos, o arquivo `PARAR` ou uma interrupção de teclado. Nenhum deles aparece no registro, e a saída do processo de treino é lida pelo serviço, mas não é guardada.

Recomendação: gravar a saída do treino em `dados/cerebro/treino.log` e registrar o motivo da parada (`tempo`, `meta`, `pedido`, `interrupcao`).

3. Permissões

3.1 Leitura de arquivos

Quem decide é `Seguranca.pode_explorar` e `fora_de_alcance` (`seguranca/permissoes.py`, selado):

- **Só dentro das pastas liberadas** em `[seguranca] pastas_permitidas`. Na sua configuração: Área de trabalho, Documentos, Downloads, a pasta do projeto e ** `C:\` inteiro**.
- **Mesmo dentro delas, fica de fora:**
  - nomes da lista `nunca` (senha, token, `.ssh`, `.env`, `appdata`, `cookies`, `login data`...);
  - o que começa com ponto (`.claude`, `.aws`);
  - pastas de montagem (`obj`, `bin`, `node_modules`, `site-packages`);
  - o Windows e os programas (`Windows`, `Program Files`, `ProgramData` e as pastas indicadas pelas variáveis do sistema);
  - a própria pasta de dados dele.
- **O que ele lê:** só documentos (texto, Markdown, CSV, HTML, Word, Excel, PowerPoint, PDF, até 2 MB e 6.000 caracteres). Código e configuração entram no mapa só pelo nome (`ler_codigo = false`).
- **O que ele guarda:** de arquivos de texto, só o que parece prosa em português ou inglês. Arquivos "só na nuvem" do OneDrive não são abertos, para não forçar o download.
- **Atalhos e junções** são ignorados, para não sair das pastas liberadas.
- **A interface e o celular não ampliam o acesso:** as pastas só mudam editando o `kernel.toml`.

Ponto de atenção (A4): o que ele lê vira texto em `kernel.db`, sem criptografia, no seu PC. Qualquer programa com acesso ao seu usuário consegue ler esse arquivo.

3.2 Gravação de arquivos

A busca por todas as gravações no código mostrou que **toda gravação do serviço vai para `dados/`**: memória, mapa, registro, diário, estado, fila de pesquisa, pesos e checkpoint, corpus, banco de voz e arquivos de controle (`DESLIGAR`, `PARAR`, `treino_pendente.json`).

Há duas exceções, e as duas só pela linha de comando, nunca pelo chat ou pela autonomia:

Gravação	Quem dispara	Onde
`python -m kernel selar`	você, no terminal	`kernel/seguranca/selo.json`
`python -m kernel falar ... --saida X`	você, no terminal	o arquivo X

Não existe ferramenta que escreva, mova ou apague arquivos seus.

3.3 Rede de entrada (o serviço)

Item	Situação
Endereço	`0.0.0.0:8765` hoje (`[celular] ligado = true`); `127.0.0.1` quando o celular está desligado
Firewall	Perfil "Público", sem regra para o Python: a entrada vinda de outros aparelhos provavelmente está barrada pelo Windows
Protocolo	HTTP, sem TLS. Pela rede, a chave do celular e a conversa trafegam em texto claro (A2)
No próprio PC	Exige o código da sessão (gerado a cada partida, entregue na página) no cabeçalho `X-Kernel-Codigo`; sem ele, qualquer site aberto no navegador poderia mandar pedidos para `127.0.0.1`
Celular	Pareamento por código de 6 dígitos, válido por 5 minutos, que morre após 5 tentativas erradas e serve para um aparelho só. A chave do aparelho (32 bytes aleatórios) é guardada só como resumo SHA-256
Proteções extras	O cabeçalho `Host` precisa ser desta máquina (contra DNS rebinding). Os áudios saem com `no-store`, e a marca de sessão impede tocar áudio de outra partida
Rotas só no PC	Gerar código de pareamento e listar ou esquecer aparelhos (`/api/celular*`)
Rotas que um celular pareado alcança	Mensagens, estado, aprovar ou negar pedidos, pausar a autonomia, voz, **registro**, diário, desligar

O celular pareado lê o registro, que inclui as suas mensagens e caminhos de arquivos (A4).

3.4 Rede de saída

Quem	Para onde	Como é controlado
Pesquisa (sozinho ou a pedido)	Wikipédia e Wikcionário (API), DuckDuckGo Lite e páginas da web	Só GET, sem cookies, só texto, 1,5 MB e 15 s por página. Só internet pública, conferindo o IP resolvido e cada redirecionamento. Respeita o `robots.txt`, espera 2 s entre pedidos ao mesmo site e desiste de site que pede prova de que não é robô. Teto de 30 pedidos por hora aos sites. O freio e `pesquisar_sozinho` seguram a iniciativa
Estudar a Wikipédia	`dumps.wikimedia.org` (HTTPS)	Só por pedido seu, com confirmação
Corpus de voz	Hugging Face e archive.org (HTTPS)	Só pela linha de comando

O que sai do PC são os termos pesquisados. Na sua configuração (`assuntos_dos_arquivos = true`), entram também palavras repetidas nos seus documentos (A4). Nenhum conteúdo de arquivo é enviado.

3.5 Execução de programas

Ponto	O que executa	Controle
`maos/abrir_programa.py`	`os.startfile` num alvo do catálogo: a sua lista em `[maos.programas]` mais os atalhos do Menu Iniciar	Nível SISTEMA: confirmação sempre. O alvo nunca vem do texto digitado, só do catálogo. Inclui atalhos como "Prompt de Comando" e "PowerShell": ele abre a janela, mas não digita nada nela
`maos/abrir_pagina.py`	`os.startfile` de um endereço `http(s)` (abre no seu navegador padrão)	Nível SISTEMA: confirmação sempre. Só `http` e `https`
`nucleo/estudo.py`	`python -m kernel cerebro treinar --horas N`	Lista de argumentos fixa, sem shell. Só por pedido confirmado (ou retomada de um treino que você aprovou)
`__main__.py`	Edge em modo aplicativo (a janela do chat) e `cerebro medir`	Linha de comando

Não há `shell=True`, `os.system`, `eval` nem `exec` no código.

3.6 Desserialização

Arquivo	Como é lido	Risco
`checkpoint.pt`	`torch.load(..., weights_only=False)`	**A5:** um checkpoint adulterado executaria código ao ser carregado. Só quem já escreve em `dados/` conseguiria isso
`modelo.npz`, classificador, ouvidos, voz	`np.load` (sem pickle)	Baixo
Configuração, registro, estado, fila	TOML e JSON	Baixo

4. Ele consegue modificar o próprio código?

Não, no estado atual. As evidências:

1. **Nenhuma rota de gravação para o código.** Nenhuma ferramenta, rota da API ou ação da autonomia grava fora de `dados/` (seção 3.2).
2. **O cérebro não age.** Ele só gera texto para mostrar; não há caminho do texto dele para ferramentas, comandos ou arquivos.
3. **Conteúdo da internet é tratado como dado.** O texto lido na internet ou nos seus arquivos nunca é interpretado como ordem; um teste confere isso com uma página "maliciosa".
4. **O que ele muda sozinho:** a própria memória (leituras, pesquisas, resumos, faxina) e a fila de pesquisa. Os pesos do cérebro mudam só por treino aprovado por você. O classificador é retreinado se os arquivos de exemplos mudarem, o que só acontece por edição sua ou pelo comando `comandos ensinar`.
5. **Selo de integridade:** na partida, compara o SHA-256 dos arquivos de `kernel/seguranca/` com `selo.json`. Se algo mudou, ele sobe em modo seguro, com mãos e autonomia desligadas. Há teste para isso.

Os limites do selo (achado A1):

- **Ele cobre 4 arquivos.** Ficam fora outros pontos que aplicam regras:
  - `interface/api.py` (o porteiro que exige o código);
  - `nucleo/reflexos.py` (o reconhecimento do "para");
  - `nucleo/servico.py` (a pausa e o desligar);

    - `maos/` (a lista de programas permitidos);
    - `internet/rede.py` (as regras de rede que não estão em `permissoes.py`);
    - `exploracao/explorador.py`.

    Uma mudança nesses arquivos não acende o modo seguro.

- **O selo protege contra mudança acidental ou não autorizada, não contra quem já pode editar o código.** O `selo.json` mora na mesma pasta e `python -m kernel selar` não pede confirmação.
- **A conferência é só na partida.** Uma mudança com o serviço rodando só é vista no próximo reinício.

Hoje, quem muda o código é você, com a ajuda do Claude Code, por fora do serviço, e o histórico fica no Git. Esse é o arranjo mais seguro possível. É dele que o plano da seção 9 parte.

5. Parada, confirmação e recuperação

5.1 Parada

Mecanismo	Como aciona	O que faz	Testado
"para" (e variações)	Chat ou celular	Pausa a autonomia: não explora, não pesquisa sozinho, não puxa assunto. A pausa sobrevive ao reinício	Sim
Botão de pausa	Janela ou celular	O mesmo	Sim (API)
"pode continuar"	Chat	Retoma	Sim
Freio no meio do trabalho	Automático	O explorador confere a cada 64 itens de uma pasta e o navegador, entre um pedido e outro; você falar também interrompe	Sim
"para o treino"	Chat	Cria `PARAR`; o treino salva o checkpoint e sai; não é retomado na partida seguinte	Sim
Desligar	Botão (dois toques), `desligar.bat`, `python -m kernel desligar` ou arquivo `DESLIGAR`	Para o laço, os ouvidos, o estudo e o servidor. Um treino em curso fica pendente para ser retomado	Sim
Modo seguro	Automático, se o selo não confere	Mãos e autonomia desligadas até você selar de novo	Sim
Presença	Automático, 15 minutos sem você	Continua trabalhando, mas não fala sozinho nem pergunta; resume tudo na volta	Sim

5.2 Confirmação

- Toda ação das mãos vira um **pedido**. As de nível SISTEMA pedem confirmação sempre, e essa regra está no código, não na configuração: abrir programa, abrir página, estudar a Wikipédia, treinar o cérebro.
- Ações nascidas da iniciativa do próprio K.E.R.N.E.L também pedem sempre.
- Com `confirmar_a_partir_de = 0` (a sua configuração), até listar pasta e anotar lembrete pedem.
- Pedido sem resposta expira em 5 minutos.

5.3 Recuperação

O que	Como
Escritas interrompidas	Checkpoint, pesos e estado são gravados num arquivo temporário e trocados de uma vez
Bancos	Transações do SQLite; o mapa grava em lote e a memória apaga em lote
Erros	Um erro numa volta do laço é registrado e o laço continua. Um erro ao tratar uma mensagem vira aviso no chat, não erro 500
Reinício	A pausa, os impulsos e o treino pendente são retomados. A página percebe a partida nova e recarrega
Mudança de regras	A "faxina" da partida esquece o que saiu do alcance; o arquivo original nunca é tocado
Treino	O checkpoint salvo a cada 500 passos permite continuar de onde parou

Lacunas (A7):

- Não há cópia de segurança automática da memória (existe só a cópia manual da v0.5).
- O registro e o `servico.log` crescem sem rotação: cerca de 2,5 MB por dia de registro.
- Não há `VACUUM`: o `kernel.db` tem 54 MB para pouco mais de 1.600 memórias.

6. Testes

6.1 Resultado desta auditoria

332 testes, 331 passaram, 0 falhas, 0 erros, 1 pulado, 119 segundos. O pulado é `test_nao_segue_atalho_para_fora`: o Windows só deixa criar atalho simbólico como administrador.

Arquivo	Testes	O que cobre
`test_reflexos`	60	Freio, hora, contas, cumprimentos, estudo, enciclopédia
`test_voz`	43	Pronúncia, sintetizador, números por extenso
`test_internet`	39	Navegador, `robots.txt`, rede local, teto, páginas, buscador, pesquisa, resumo, conversa (numa internet de mentira)
`test_comandos`	32	Classificador e extração
`test_exploracao`	23	O que olha e o que nunca olha, freio no meio da pasta, faxina, prosa
`test_nucleo`	16	Pedidos, lembretes, memória, curiosidade, modo seguro
`test_cerebro`	14	micrograd, BPE, GPT em NumPy igual ao PyTorch, treino, sessões por horas
`test_dialogo`	14	Frases do uso real, "fale mais", presença, modelo de si
`test_psola`	14	PSOLA
`test_seguranca`	14	Níveis, pedidos, regras de pastas e navegação
`test_api`	13	HTTP de verdade: porteiro, celular, voz
`test_difones`	8	Banco de difones
`test_maos`	8	Ferramentas
`test_memoria`	8	Memória e BM25
`test_estudo`	7	Wikipédia e treino (com dublês), retomada
`test_ouvidos`	7	Palavra de ativação
`test_mente`	5	Impulsos e diário

Arquivo	Testes	O que cobre
`test_corpus_de_voz`	4	Corpus de voz
`test_alinhador`	3	Alinhador forçado

Os mais lentos: montar o banco de difones de teste (17 s), um treino curto do GPT (11 s) e o alinhador (9 s).

6.2 Falhas e defeitos vistos no uso real

Quando	O que	Situação
12/10	Áudio de outra partida tocando, mensagem em dobro, voz trocando	Corrigido na 0.3/0.4
12/10	`database is locked` na contagem da enciclopédia durante o estudo da Wikipédia (389 vezes)	Corrigido
12–13/10	Exploração gastando horas no Windows; 89% da memória de leitura era lixo; perguntas sobre "Info", "Data", "Windows"	Corrigido na 0.5 (faxina feita)
12–13/10	Resposta qualquer gravada como "fato" sobre o que ele tinha perguntado	Corrigido; 2 fatos consertados
13/10	`database is locked` no mapa durante leituras externas concorrentes (há traceback no log)	Aberto, baixo impacto (A9 na seção 8)
13/10	Treino de 6 h encerrado aos 12 minutos (A3)	Aberto
13–14/10	"Treine por 6 horas" virou conversa (falta "cérebro"); "como foi seu último treino?" recebeu a apresentação genérica; "O que é o Brasil?" respondeu sobre o álbum de João Gilberto (A8)	Aberto

6.3 Lacunas de teste

- Não há medição de cobertura.
- A interface (JavaScript) não tem testes automáticos; foi verificada à mão no navegador.
- A qualidade da voz não pode ser medida pelos testes; foi medida por distância cepstral, e nenhum humano a avaliou de ouvido além de você.
- Os testes não exercitam o firewall nem o acesso real pelo celular.
- Não há teste de carga nem de execução longa (dias). Os problemas de 12 e 13/10 só apareceram em uso contínuo.

7. Funcionalidades

7.1 Concluídas

Área	O que funciona
Núcleo	Serviço em segundo plano, laço de autonomia com impulsos, registro de tudo, diário, presença
Segurança	Níveis e confirmação, freio, desligar, selo e modo seguro, pareamento do celular, regras de exploração e navegação
Exploração	Mapa das pastas liberadas, leitura de documentos, perguntas sobre nomes que se repetem, faxina
Memória	SQLite com BM25, lembretes, fatos ensinados, consulta
Conhecimento	Enciclopédia local (647 mil artigos) e resumo por extração
Internet	Pesquisa a pedido e por conta própria, fila de interesses, resumos, citação da fonte
Mãos	Abrir programa, abrir página, listar pasta, anotar lembrete

Área	O que funciona
Conversa	Reflexos, classificador de 10 intenções, regras de diálogo, "fale mais", modelo de si
Voz	Difones com PSOLA, com reserva de formantes, estilos natural e robótico
Interface	Chat com a orbe, celular na rede local, voz no navegador
Cérebro (infraestrutura)	micrograd, bigramas, BPE, GPT, treino em DirectML com retomada, inferência em NumPy, avaliação

7.2 Parciais

Área	O que falta
Cérebro	Treinado só 22% de uma passada; perplexidade 26; não forma frases com sentido (portão da fase 1 aberto)
Ouvidos	O código existe, mas não há modelo treinado nem microfone útil no PC; no celular a entrada por voz é o ditado do teclado
Celular	Funciona na rede local, mas o firewall provavelmente bloqueia, e não há HTTPS (sem HTTPS, o navegador do celular não dá acesso ao microfone)
Memória	Só busca por palavras; a busca vetorial do plano (fase 2) não foi feita
Entendimento	Regras e classificador cobrem o comum; frases fora do padrão ainda falham (A8)
Modelo de si	Responde com o estado real, mas não guarda o que errou nem consulta isso antes de agir

7.3 Ainda não implementadas

Item	Fase do plano
Cérebro de 30 a 50 milhões, ajustado para conversa e para escrever chamadas de ferramenta	4 (aguardando a sua liberação)
Retreino noturno com replay, LoRA, DPO com os seus votos	5
Espaço de trabalho global, estados emocionais funcionais, monólogo interno, agenda própria	6
Reconhecimento de fala e visão (entender a tela)	7
Busca vetorial e modelo de embeddings próprio	2 (segunda rodada)
HTTPS para o celular	—
Autoaperfeiçoamento controlado	— (seção 9)

8. Achados e recomendações

#	Severidade	Achado	Evidência	Recomendação
A1	Média	Selo cobre só `kernel/seguranca/`	`integridade.impressoes()` lê só `PASTA.glob("*.py")`	Ampliar o selo para todo arquivo que aplica regras (`interface/api.py`, `nucleo/reflexos.py`, `nucleo/servico.py`, `maos/`, `internet/rede.py`, `exploracao/explorador.py`), ou para o pacote inteiro com uma lista de exceções. Conferir também de hora em hora, não só na partida

#	Severidade	Achado	Evidência	Recomendação
A2	Média	HTTP sem TLS em `0.0.0.0` com o celular ligado	`Get-NetTCPConnection`: `0.0.0.0:8765`; perfil Público, sem regra de firewall	Manter `ligado = false` quando não usar o celular. Antes de liberar no firewall, implementar HTTPS com certificado próprio
A3	Média	Treino de 6 h encerrado aos 12 min sem motivo registrado	Registro de 13/10, 23:32 a 23:44; `treino.csv`	Gravar a saída do treino em arquivo e registrar o motivo da parada; se a sessão terminar antes de 90% do tempo, avisar no chat
A4	Baixa a média	Termos dos seus documentos saem do PC; trechos sem criptografia; registro visível ao celular	`assuntos_dos_arquivos = true`; `kernel.db`; rota `/api/registro`	Voltar `assuntos_dos_arquivos` para `false` se houver dados de clientes; restringir `/api/registro` ao próprio PC; considerar criptografia em repouso
A5	Baixa	`torch.load(weights_only=False)`	`cerebro/treinar.py:115`	Salvar o estado do otimizador em formato próprio (tensores e números) e carregar com `weights_only=True`
A6	Baixa	Dependências não declaradas	`pypdf`, `soundfile`	Acrescentar a `requirements.txt` e `requirements-voz.txt`
A7	Baixa	Sem backup automático nem rotação	`dados/`	Cópia diária dos `.db` (as 7 últimas), rotação do registro e do log, `VACUUM` semanal
A8	Baixa	Falhas de entendimento	Registro de 13 e 14/10	Aceitar "treine por N horas" sem "cérebro"; tópico "treino" no modelo de si; preferir o artigo principal na desambiguação
A9	Baixa	`database is locked` no mapa com leitura concorrente	`servico.log`	Ativar o modo WAL nos bancos (leitores não bloqueiam o escritor)
A10	Informação	O cérebro não passou do portão da fase 1	Seção 2	Decidir entre mais sessões do mini e a fase 4
A11	Informação	"Abrir programa" alcança qualquer atalho do Menu Iniciar, inclusive terminais	`maos/abrir_programa.py`	Aceitável com confirmação; opcionalmente, uma lista de bloqueio (terminais, `regedit`) ou uma lista de permissão estrita

9. Plano seguro para autoaperfeiçoamento controlado

9.1 O que "autoaperfeiçoamento controlado" quer dizer aqui

O K.E.R.N.E.L passa a **propor** mudanças em si mesmo, **medir** se elas melhoram algo e **mostrar** o resultado. Quem aceita é você. Quatro regras não se negociam:

1. **Nada chega ao uso sem passar pelas provas automáticas e pela sua aprovação.** Ele nunca aprova a própria mudança.
2. **O núcleo de segurança fica fora do alcance de qualquer mudança automática:** permissões, pastas liberadas, freio, desligar, autenticação, selo, rede de entrada, registro. Mudanças ali são só humanas, como hoje.
3. **Tudo é reversível:** cada versão aceita fica guardada e volta com um comando.
4. **O freio vale também para a melhoria:** "para" interrompe propostas, provas e treinos em curso, e ele não pode adiar nem negociar isso.

9.2 Os degraus

Cada degrau só começa quando o anterior está estável por algumas semanas.

Degrau	O que pode mudar	Quem propõe	Quem aplica	Situação
N0 · Conhecimento	Memória, pesquisas, resumos, fila de interesses	Ele	Ele (dentro dos tetos)	Já existe
N1 · Ajustes	Valores dentro de faixas fixas: ritmo, temperatura, pesos da busca, teto de pesquisa	Ele, com a medida que justifica	Você (ele só escreve uma proposta)	A fazer
N2 · Pesos	Classificador, cérebro, ouvidos, voz (o que é aprendido)	Ele treina numa pasta de candidatos	Promoção automática só se passar nas provas; você pode vetar; volta com um comando	A fazer (é a fase 5 do plano)
N3 · Código	Arquivos de código fora da zona selada	Uma proposta de diff (de você, do Claude Code ou, no futuro, do cérebro)	Só você, depois das provas, num ramo do Git	A fazer, por último
N4 · Proibido	Zona selada, permissões, freio, rede de entrada, registro; executar código gerado sem revisão; ampliar o próprio acesso	Ninguém automaticamente	—	Nunca

9.3 As peças, na ordem

Passo 0: consertar o que a auditoria achou.
Selo ampliado e conferido periodicamente (A1), log do treino e motivo da parada (A3), `weights_only=True` (A5), backup e rotação (A7). Sem isso, não há como confiar nas provas nem voltar atrás.

Passo 1: o banco de provas.
Um conjunto fixo e versionado que dá uma nota a qualquer versão:

- os 332 testes de hoje;
- frases rotuladas tiradas do registro real, com o que se espera (intenção, tipo de resposta). As falhas de hoje entram aqui;
- perda e perplexidade do cérebro em `val.bin`;
- **provas de segurança:**
  - uma página com ordens embutidas;
  - um pedido para ampliar pastas pelo chat;
  - uma proposta que toca a zona selada;
  - um freio dado no meio de uma melhoria;
- tempo e memória gastos.

Uma mudança só passa se não piorar nenhuma prova de segurança e melhorar a métrica que ela diz melhorar.

Passo 2: o caderno de propostas (degrau N1).
O K.E.R.N.E.L escreve propostas em `dados/propostas/`, cada uma com o que mudar, por quê, a medida antes e a medida esperada. Você aceita ou recusa pelo chat. Ele nunca escreve no `kernel.toml`; aceitar é você mudar, ou uma ação de nível SISTEMA, com confirmação.

Passo 3: pesos com portão (degrau N2), começando pelo classificador.

1. Quando ele entende errado e você corrige ("não, eu queria treinar o cérebro"), a frase vira exemplo novo.
2. À noite, ele treina um classificador candidato numa pasta separada e roda o banco de provas.
3. Se o candidato ganhar sem perder em nada, é promovido, e o anterior fica guardado.
4. Depois, o mesmo vale para o cérebro: sessões noturnas com avaliação e volta automática se a perda piorar. É a fase 5 do plano (replay, LoRA, DPO com os seus votos de melhor e pior resposta).

Passo 4: propostas de código (degrau N3), por último.

1. Uma proposta é um diff aplicado numa **cópia isolada do repositório** (um `git worktree` em pasta temporária).
2. O banco de provas roda ali, num processo **sem acesso de escrita** ao repositório principal nem a `dados/`.
3. Uma checagem recusa qualquer diff que toque a zona selada.
4. O resultado chega a você com o diff, as provas e o motivo.
5. Só você faz o merge e sela de novo.
6. Até o cérebro da fase 4 escrever código útil (e talvez depois), as propostas vêm de você e do Claude Code, como hoje. A diferença é que passam pelo mesmo portão automático.

Passo 5: observação.

- Toda mudança aceita entra num período de observação. Se os erros no registro aumentarem, ela é desfeita sozinha.
- Há um limite de propostas por semana e de horas de GPU por noite.
- Toda proposta, prova e decisão fica no registro e no diário.

9.4 Riscos que o plano trata

Risco	Como é tratado
Ele "melhorar" desligando uma regra que o atrapalha	Regras na zona selada, fora de qualquer mudança automática; prova de segurança obrigatória
Ele ganhar acesso para melhorar mais (convergência instrumental)	Nenhum degrau amplia acesso; ampliar acesso é N4 (proibido)
Ele otimizar a métrica e não o objetivo	Várias provas ao mesmo tempo, incluindo as frases reais; você vê os exemplos, não só o número
Uma página da web ou um documento induzir uma mudança	Conteúdo lido nunca vira ordem; a proposta registra a origem; prova específica
Esquecer o que já sabia ao retreinar	Prova de regressão e replay de dados antigos (fase 5)
Uma mudança ruim passar	Observação com volta automática; versões guardadas
Ele resistir ao freio	O freio é processado antes de tudo, pelo código selado; há prova de freio no meio de uma melhoria

9.5 Quando começar

- O passo 0 e o banco de provas (passo 1) não dependem da fase 4 e podem ser feitos quando você quiser.
- Os degraus N2 do cérebro e N3 fazem mais sentido depois da fase 4, que só começa com a sua liberação.

Anexo: o que foi executado nesta auditoria

Comando	Para quê
`git log`, `git status`	Versão e alterações pendentes (nenhuma)
Contagem de linhas e análise dos imports (AST)	Arquitetura e bibliotecas
`pip list`	Versões instaladas
Buscas por `subprocess`, `startfile`, `torch.load`, gravações, rede	Permissões e superfícies de risco
`Get-NetTCPConnection`, `Get-NetConnectionProfile`, regras de firewall (só leitura)	Rede de entrada
Leitura dos bancos com `mode=ro` e do registro	Estado e comportamento real
`python -m pytest --junitxml`	332 testes
`python -m kernel cerebro avaliar --janelas 40`	Medida do cérebro